

Hands-on Deep Learning - Aula 2

Redes Neurais e Redes Convolucionais

Hugo Oliveira¹², Camila Laranjeira²³

¹Instituto de Matemática e Estatística (IME)
Universidade de São Paulo

²Interest Group in Pattern Recognition and Earth Observation (PATREO)
Universidade Federal de Minas Gerais

³Programa de Pós-Graduação em Ciência da Computação (PPGCC)
Universidade Federal de Minas Gerais

28 de Julho, 2018

Agenda



1 Origem

2 Fundamentos

- Tensores
- Classificação Linear
- Perceptron
- Otimização
- Redes Neurais Artificiais
- Treinamento

Agenda



1 Origem

2 Fundamentos

- Tensores
- Classificação Linear
- Perceptron
- Otimização
- Redes Neurais Artificiais
- Treinamento

O Cérebro



- Redes Neurais foram originalmente inspiradas no cérebro humano
- Sua unidade básica é o neurônio, composto por 3 partes principais:
 - Dendritos: Recebem sinais de outros neurônios conectados a ele
 - Corpo: Responsável por “processar” a informação
 - Axônio: Transmite o sinal processado para outros neurônios. A ativação do axônio depende da força sináptica do sinal.

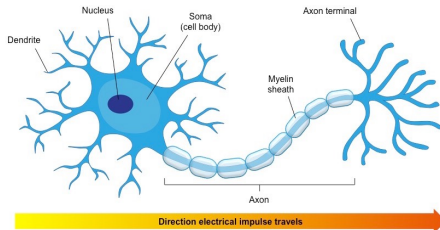


Figura: Representação de um neurônio [1]

O Cérebro

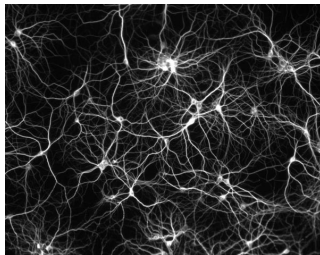


Figura: Representação da malha neural [2]

- Os neurônios se conectam uns aos outros, criando uma grande malha.
- Responsável pelo processamento de todas as funções do corpo.



Representação Artificial do Cérebro

- Redes neurais artificiais são sistemas compostos de neurônios artificiais **inspirados** no neurônio biológico.

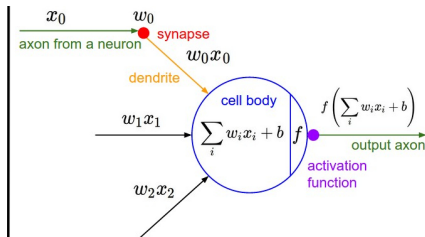
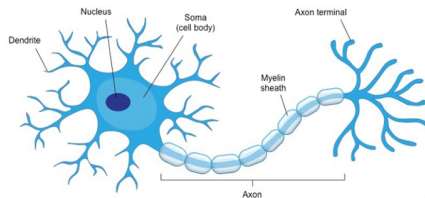
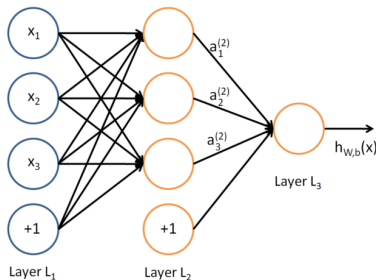
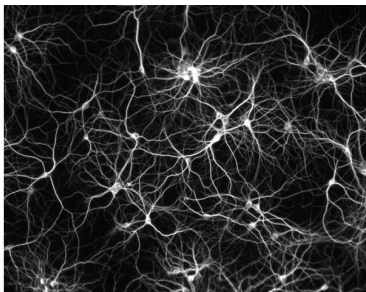


Figura: Comparando os neurônios biológico e artificial (Perceptron).

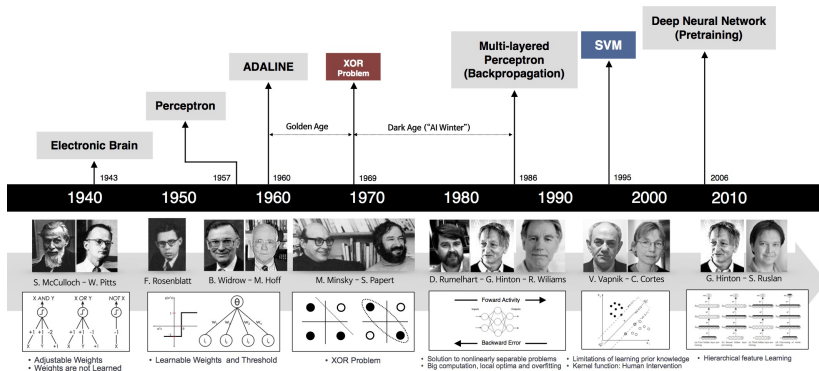
Representação Artificial do Cérebro



- Redes neurais artificiais são sistemas compostos de neurônios artificiais **inspirados** no neurônio biológico.
 - Similarmente combinados na forma de uma malha neural.
 - Esse modelo é **muito limitado** se comparado com o cérebro real.



Representação Artificial do Cérebro



Agenda



1 Origem

2 Fundamentos

- Tensores
- Classificação Linear
- Perceptron
- Otimização
- Redes Neurais Artificiais
- Treinamento

Tensores



- Sintaxe parecida com ndarrays (Numpy)
- Armazena tanto os inputs e labels quanto os pesos e viéses das NNs
- Funções para tradução de ndarrays para tensores e vice-versa

Tensores



Data type	dtype	CPU tensor
32-bit floating point	<code>torch.float32</code> or <code>torch.float</code>	<code>torch.FloatTensor</code>
64-bit floating point	<code>torch.float64</code> or <code>torch.double</code>	<code>torch.DoubleTensor</code>
16-bit floating point	<code>torch.float16</code> or <code>torch.half</code>	<code>torch.HalfTensor</code>
8-bit integer (unsigned)	<code>torch.uint8</code>	<code>torch.ByteTensor</code>
8-bit integer (signed)	<code>torch.int8</code>	<code>torch.CharTensor</code>
16-bit integer (signed)	<code>torch.int16</code> or <code>torch.short</code>	<code>torch.ShortTensor</code>
32-bit integer (signed)	<code>torch.int32</code> or <code>torch.int</code>	<code>torch.IntTensor</code>
64-bit integer (signed)	<code>torch.int64</code> or <code>torch.long</code>	<code>torch.LongTensor</code>

Figura: Tipos de Tensores.

Tensoros



Demo - Tensors

Tensors.ipynb

Classificação Linear



- **Mapeamento:** Uma regra que, para cada elemento do conjunto X , provê o elemento correspondente do conjunto Y .

$$f : X \rightarrow Y$$

- Um mapeamento nada mais é do que uma **função**.

$$y = f(x)$$

Classificação Linear

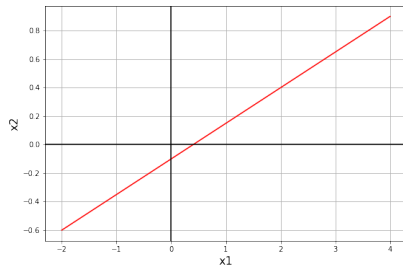


- Equação da Reta

$$w_1 x_1 + w_2 x_2 + b = 0$$

- Um peso w_i por dimensão

- Um viés b



Classificação Linear



- Equação da Reta: Pode servir como uma função de mapeamento.

$$f(x; W, b) = w_1 x_1 + w_2 x_2 + b$$

- Em outras palavras: Dados os **parâmetros** $W = \{w_1, w_2\}$ e b de uma reta, é possível mapear uma entrada $X = \{x_1, x_2\}$ para a saída $f(x; W, b)$.

- Representação matricial

$$\underbrace{\begin{bmatrix} w_1 & w_2 \end{bmatrix}}_W + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}_{X^i} + b = \text{score} \quad f(X^i; W, b)$$

Classificação Linear



- Equação da Reta:

$$f(x; W, b) = w_1 x_1 + w_2 x_2 + b$$

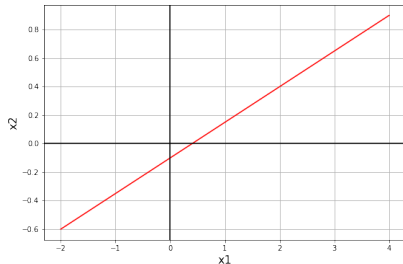
- Vamos supor uma reta fixa:

$$f(x; W, b) = -1x_1 + 4x_2 + 0.4$$

- Representação matricial

$$\underbrace{\begin{bmatrix} -1 & 4 \end{bmatrix}}_W + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \underbrace{0.4}_b = \text{score}$$

$f(X^i; W, b)$



Classificação Linear

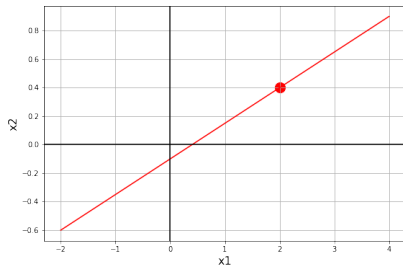


- Vamos supor uma reta fixa:

$$f(x; W, b) = -1x_1 + 4x_2 + 0.4$$

- Para $X = \{2.0, 0.4\}$

$$\underbrace{\begin{bmatrix} -1 & 4 \end{bmatrix}}_W \underbrace{\begin{bmatrix} 2 \\ 0.4 \end{bmatrix}}_{X^i} + \underbrace{0.4}_b = \underbrace{0}_{f(X^i; W, b)}$$



- $f(x; W, b) = 0$: ponto sobre a reta.

Classificação Linear

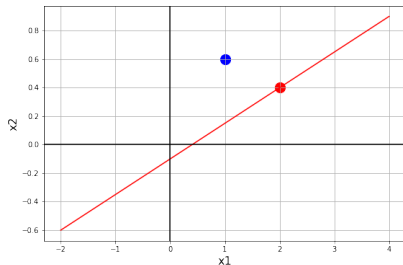


- Vamos supor uma reta fixa:

$$f(x; W, b) = -1x_1 + 4x_2 + 0.4$$

- Para $X = \{1.0, 0.6\}$

$$\underbrace{\begin{bmatrix} -1 & 4 \end{bmatrix}}_W + \underbrace{\begin{bmatrix} 1 \\ 0.6 \end{bmatrix}}_{X^i} + 0.4 = \begin{bmatrix} 1.8 \end{bmatrix} \quad f(X^i; W, b)$$



- $f(x; W, b) > 0$: ponto acima da reta.

Classificação Linear

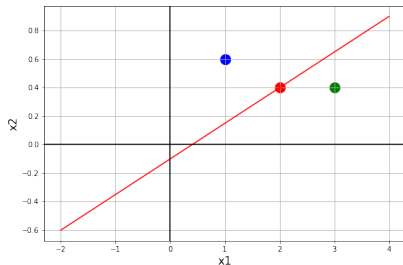


- Vamos supor uma reta fixa:

$$f(x; W, b) = -1x_1 + 4x_2 + 0.4$$

- Para $X = \{3.0, 0.4\}$

$$\underbrace{\begin{bmatrix} -1 & 4 \end{bmatrix}}_W \underbrace{\begin{bmatrix} 3 \\ 0.4 \end{bmatrix}}_{X^i} + \underbrace{0.4}_b = \underbrace{-1}_{f(X^i; W, b)}$$



- $f(x; W, b) < 0$: ponto abaixo da reta.

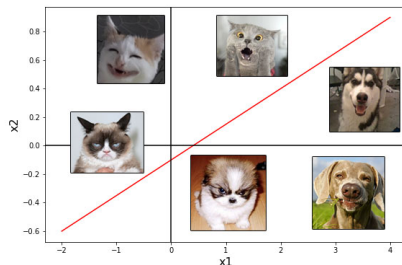
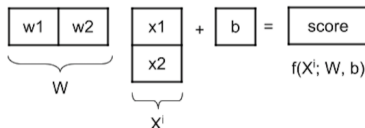
Classificação Linear



- Equação da Reta

$$f(x; W, b) = w_1 x_1 + w_2 x_2 + b$$

- Representação matricial

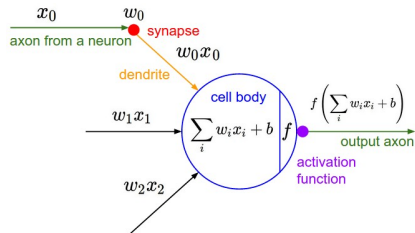


- Em termos gerais, o treinamento de um classificador linear busca os **parâmetros** W e b para compor a **fronteira de decisão** entre as classes.

Perceptron



- Unidade elementar das redes neurais.
- Modelo matemático que mapeia suas entradas para uma saída.
- Exemplo: um ponto em 3 dimensões $X = \{x_0, x_1, x_2\}$ vai compor um perceptron com 3 entradas e 1 saída.



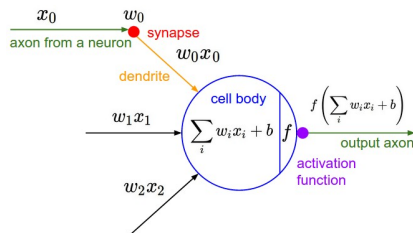
Perceptron



- Mapeamento linear

$$z = \sum_{i=0}^N w_i x_i + b$$

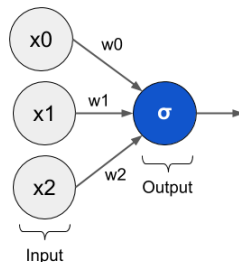
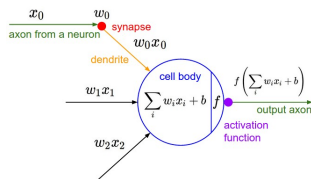
- Seguido de uma função de ativação f



Perceptron



- Pode ser representado como uma rede neural de 2 camadas
 - Camada 1: Entrada. A dimensão dessa camada é definida de acordo com o dado.
 - Camada 2: Saída. Dimensão diretamente ligada ao problema. Por exemplo, para um problema de classificação a dimensão equivale ao número de classes.

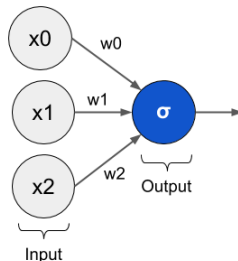
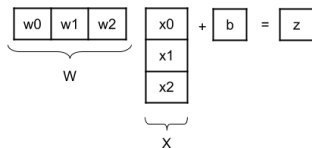


Perceptron

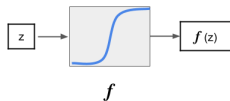


$$z = \sum_{i=1}^N w_i x_i + b$$

- W é o peso de cada ligação
- b é o bias

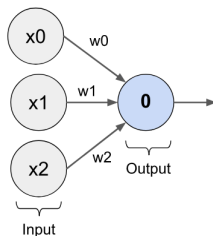
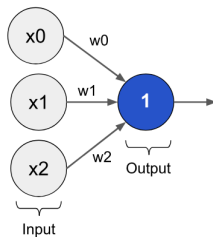


Ativações



- f é a função de ativação
- Por exemplo:

$$f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

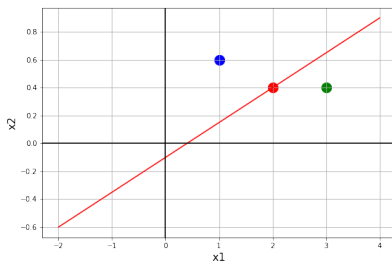


Ativações



- $f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases}$

- $z = -1x_1 + 4x_2 + 0.4$



- $z = -1 \times 1 + 4 \times 0.6 + 0.4 = 1.8$

- $f(z) = 1$

- $z = -1 \times 3 + 4 \times 0.4 + 0.4 = -1$

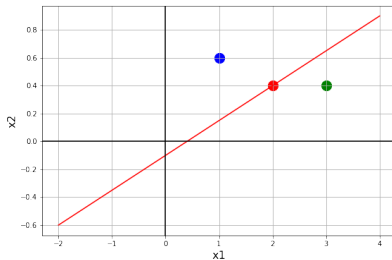
- $f(z) = 0$

Ativações



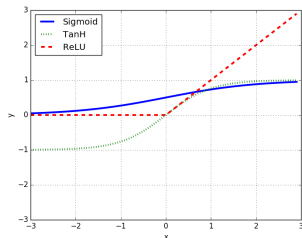
- $f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases}$

- $z = -1x_1 + 4x_2 + 0.4$



- Essa função de ativação permite maior controle sobre a saída do perceptron. A saída z que antes poderia assumir qualquer valor $z = \{-\infty, \dots, \infty\}$ agora está limitada a $z = \{0, 1\}$

Ativações



- Sigmoid (σ)
$$f(x) = \frac{1}{1+e^{-x}}$$
- TanH
$$f(x) = 2\sigma(2x) - 1$$
- ReLU
$$f(x) = \max(0, x)$$

- Entre outras
 - Threshold, Leaky ReLU, Maxout, Linear, etc.
- Em redes com mais de uma camada escondida, a adição de uma função de ativação não linear permite que a rede modele dependências não lineares.

Ativações

- Softmax



- Geralmente SoftMax é usada como função de ativação na última camada de Redes para classificação com 2 ou mais categorias.
- Isso porque ela é normalizada para representar uma distribuição de probabilidade.

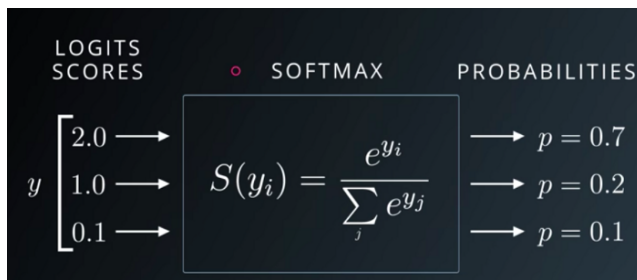


Figura: Slide da Udacity sobre Softmax.

Perceptron como Classificador Linear



- Função de mapeamento do neurônio

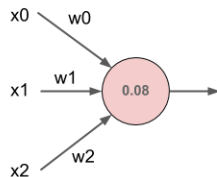
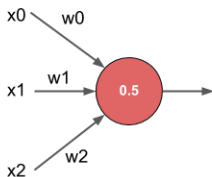
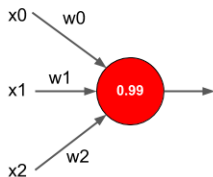
$$f\left(\sum_{i=0}^N w_i x_i + b\right)$$

- O neurônio faz um mapeamento linear, permitindo que seja possível treiná-lo para encontrar os parâmetros da fronteira de decisão tal qual um classificador linear para um problema de classificação binária $Y = \{0, 1\}$.

Perceptron como Classificador Linear



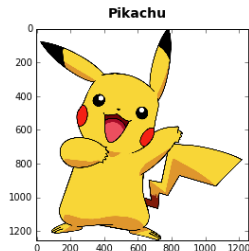
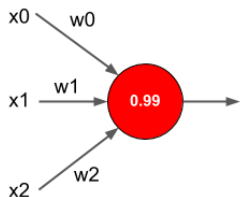
- De forma abstrata, um perceptron interpreta as suas entradas, e libera uma ativação com uma determinada força.
- Assumindo um perceptron com ativação sigmoide (varia entre 0 e 1), temos:



Perceptron como Classificador Linear



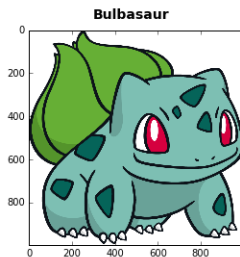
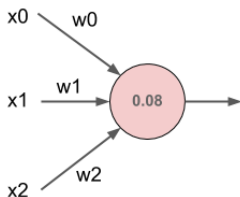
- Exemplo: você treinou um perceptron para fazer uma classificação binária, associando um rótulo numérico para cada classe $Y = \{1 : \text{Pikachu}, 0 : \text{Bulbasaur}\}$. Quanto mais próxima a ativação for de 1, mais certa o perceptron tem da classe "Pikachu".



Perceptron como Classificador Linear



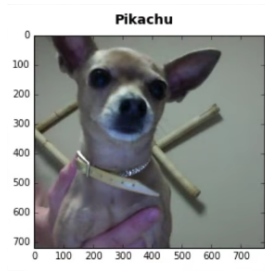
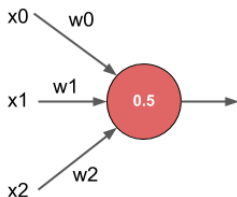
- Exemplo: você treinou um perceptron para fazer uma classificação binária, associando um rótulo numérico para cada classe $Y = \{1 : \textit{Pikachu}, 0 : \textit{Bulbasaur}\}$. Quanto mais próxima a ativação for de 0, mais certa o perceptron tem da classe "Bulbasaur".



Perceptron como Classificador Linear



- Exemplo: você treinou um perceptron para fazer uma classificação binária, associando um rótulo numérico para cada classe $Y = \{1 : \textit{Pikachu}, 0 : \textit{Bulbasaur}\}$. Valores intermediários indicam que a imagem está na **zona de dúvida** da classificação.



Perceptron



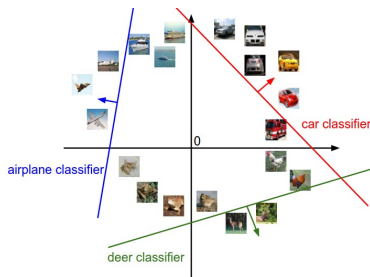
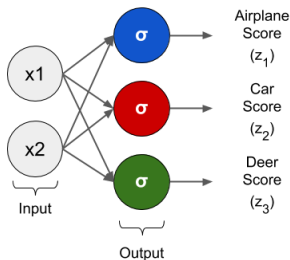
Treinando um Perceptron

Perceptron.ipynb

Perceptron como Classificador Linear



- Para problemas com múltiplas classes, é necessário especializar cada perceptron em uma classe específica.
- Cada perceptron vai otimizar os parâmetros da fronteira de decisão da classe que ele deve se especializar.



Perceptron como Classificador Linear



- Na representação matricial, isso significa que a matriz de pesos W tem a forma $K \times D$, sendo K o número de classes e D a dimensão do dado. E o vetor de bias b possui a forma $K \times 1$, apenas um bias por classe.
- Em outras palavras, cada linha das matrizes W e b produzem o score de uma classe específica.

w11	w12
w21	w22
w31	w32

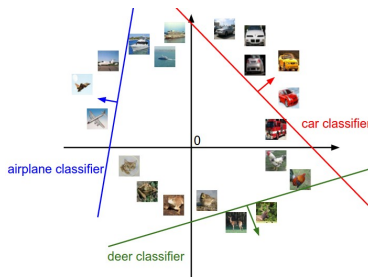
x1
x2

+

b1
b2
b3

=

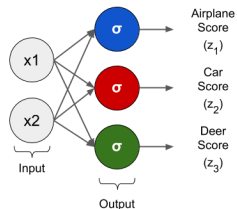
z ₁
z ₂
z ₃



Perceptron como Classificador Linear



- Para problemas com múltiplas classes



w11	w12
w21	w22
w31	w32

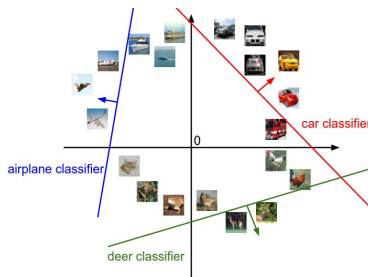
x_1
x_2

 $+$

b1
b2
b3

 $=$

z1
z2
z3

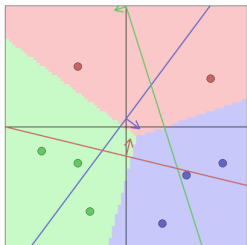


Perceptron como Classificador Linear



Datapoints are shown as circles colored by their class (red/green/blue). The background regions are colored by whichever class is most likely at any point according to the current weights. Each classifier is visualized by a line that indicates its zero score level set. For example, the blue classifier computes scores as $W_{0,0}x_0 + W_{0,1}x_1 + b_0$ and the blue line shows the set of points (x_0, x_1) that give score of zero. The blue arrow draws the vector $(W_{0,0}, W_{0,1})$, which shows the direction of score increase and its length is proportional to how steep the increase is.

Note: you can drag the datapoints.



Parameters W, b are shown below. The value is in **bold** and its gradient (computed with backprop) is in *red*, *italic* below. Click the triangles to control the parameters.

$W[0,0]$	$W[0,1]$	$b[0]$
▲ 1.12 0.02	▲ 0.84 0.02	▲ 0.06 0.22
$W[1,0]$	$W[1,1]$	$b[1]$
▼ -0.95 0.06	▼ 0.30 -0.10	▼ 0.30 -0.11
$W[2,0]$	$W[2,1]$	$b[2]$
▼ 0.35 -0.06	▼ -1.44 0.06	▼ 0.35 -0.11

Step size: 0.10000

Single parameter update

Start repeated update

Stop repeated update

Randomize parameters

Visualization of the data loss computation. Each row is loss due to one datapoint. The first three columns are the 2D data x_i and the label y_i . The next three columns are the three class scores from each classifier $f(x_i; W, b) = Wx_i + b$ (E.g. $s[0] = x[0] * W[0,0] + x[1] * W[0,1] + b[0]$). The last column is the data loss for a single example, L_i .

$x[0]$	$x[1]$	y	$s[0]$	$s[1]$	$s[2]$	L
0.50	0.40	0	0.95	-0.05	-0.05	0.00
0.80	0.30	0	1.21	-0.37	0.20	0.00
0.30	0.80	0	1.06	0.26	-0.70	0.19
-0.40	0.30	1	-0.14	0.77	-0.22	0.09
-0.30	0.70	1	0.31	0.80	-0.74	0.51
-0.70	0.20	1	-0.56	1.03	-0.18	0.00
0.70	-0.40	2	0.51	-0.49	1.17	0.34
0.80	-1.15	2	-0.00	-0.81	2.28	0.00
-0.40	-0.50	2	-0.81	0.53	0.93	0.60
mean:						0.19

Total data loss: 0.19

Regularization loss: 0.52

Total loss: 0.71

L2 Regularization strength: 0.10000

Multiclass SVM loss formulation:

● Weston Watkins 1999

● One vs. All

● Structured SVM

● Softmax

Figura: Demo interativa da Universidade de Stanford (CS231n):

<http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/>

Funções de Perda



- Primeiro precisamos definir uma medida de qualidade
- Isso é obtido através da função de perda (*loss function*)
 - Também conhecida como função de custo
- A escolha da *loss function* vai depender do problema sendo abordado

Funções de Perda

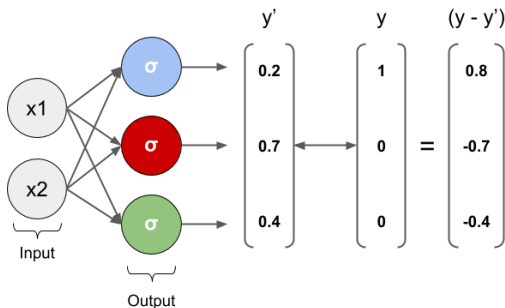


- *“A função de custo reduz todos os aspectos bons e ruins de um sistema complexo a um único número, um valor escalar, o que permite ranquear e comparar as soluções candidatas.”*
- *“Se fizermos uma escolha ruim de função de custo e os resultados obtidos não forem satisfatórios, é nossa culpa por não especificar bem nosso objetivo.”*
- Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks, Pág 155, 1999.

Funções de Perda (Exemplo)



- y' representa a saída da rede dada a entrada x
 - $f(x; \mathbf{W}, \mathbf{b})$
- y é o rótulo da amostra x
- $(y - y')$ é a função de custo

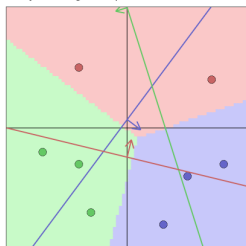


Perceptron como Classificador Linear



Datapoints are shown as circles colored by their class (red/green/blue). The background regions are colored by whichever class is most likely at any point according to the current weights. Each classifier is visualized by a line that indicates its zero score level set. For example, the blue classifier computes scores as $W_{0,0}x_0 + W_{0,1}x_1 + b_0$ and the blue line shows the set of points (x_0, x_1) that give score of zero. The blue arrow draws the vector $(W_{0,0}, W_{0,1})$, which shows the direction of score increase and its length is proportional to how steep the increase is.

Note: you can drag the datapoints.



Parameters W, b are shown below. The value is in **bold** and its gradient (computed with backprop) is in *red*, *italic* below. Click the triangles to control the parameters.

$W[0,0]$	$W[0,1]$	$b[0]$
▲ 1.12 0.02	▲ 0.84 0.02	▲ 0.06 0.22
$W[1,0]$	$W[1,1]$	$b[1]$
▼ -0.95 0.06	▼ 0.30 -0.10	▼ 0.30 -0.11
$W[2,0]$	$W[2,1]$	$b[2]$
▼ 0.35 -0.06	▼ -1.44 0.06	▼ 0.35 -0.11

Step size: 0.10000

Single parameter update

Start repeated update

Stop repeated update

Randomize parameters

Visualization of the data loss computation. Each row is loss due to one datapoint. The first three columns are the 2D data x_i and the label y_i . The next three columns are the three class scores from each classifier $f(x_i; W, b) = Wx_i + b$ (E.g. $s[0] = x[0] * W[0,0] + x[1] * W[0,1] + b[0]$). The last column is the data loss for a single example, L_i .

$x[0]$	$x[1]$	y	$s[0]$	$s[1]$	$s[2]$	L
0.50	0.40	0	0.95	-0.05	-0.05	0.00
0.80	0.30	0	1.21	-0.37	0.20	0.00
0.30	0.80	0	1.06	0.26	-0.70	0.19
-0.40	0.30	1	-0.14	0.77	-0.22	0.09
-0.30	0.70	1	0.31	0.80	-0.74	0.51
-0.70	0.20	1	-0.56	1.03	-0.18	0.00
0.70	-0.40	2	0.51	-0.49	1.17	0.34
0.80	-1.15	2	-0.00	-0.81	2.28	0.00
-0.40	-0.50	2	-0.81	0.53	0.93	0.60
mean:						0.19

Total data loss: 0.19

Regularization loss: 0.52

Total loss: 0.71

L2 Regularization strength: 0.10000

Multiclass SVM loss formulation:

● Weston Watkins 1999

● One vs. All

● Structured SVM

● Softmax

Figura: Demo interativa da Universidade de Stanford (CS231n):

<http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/>

Funções de Perda

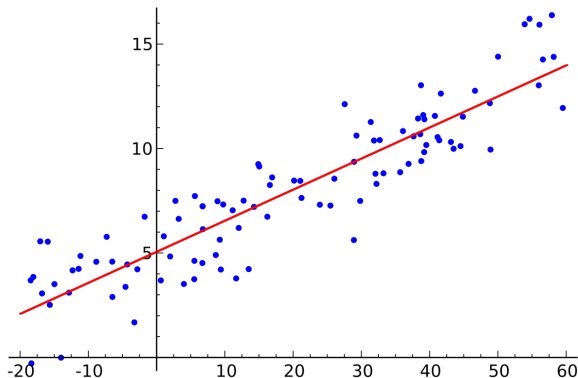


- CrossEntropyLoss – Classificação
- NLLLoss – Classificação
- MSELoss – Regressão/Reconstrução
- L1Loss – Regressão/Reconstrução
- KLDivLoss – Reconstrução
- HingeEmbeddingLoss – Aprendizado Semi-Supervisionado

Funções de Perda



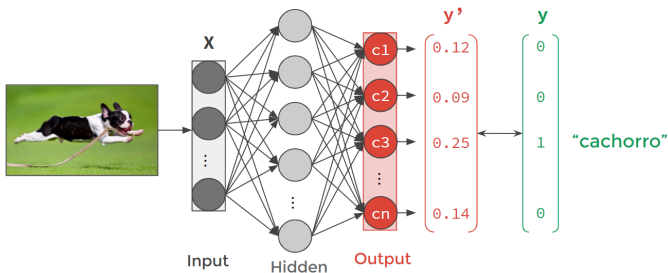
- Na **regressão** faz sentido adotar métricas como a distância euclidiana ou o erro quadrático.



Funções de Perda



- Já na **classificação**, estamos comparando duas distribuições. Isso requer, portanto, uma métrica como a *cross-entropy* que define a entropia entre distribuições.



Funções de Perda



Funções de Perda

Funções de Perda diferentes recebem **inputs e targets com dimensões diferentes**

Funções de Perda



Demo - Loss Functions

Loss_Functions.ipynb

Otimização



- O nosso objetivo então é minimizar essa função de custo
- Como pôde ser visto, a função de custo $\mathcal{L}(W, b; x, y)$ é baseada nas variáveis da rede:
 - pesos W
 - bias b
- Logo, a minização dessa função se dá ao mudar essas variáveis baseado no erro que ela gera

Otimização

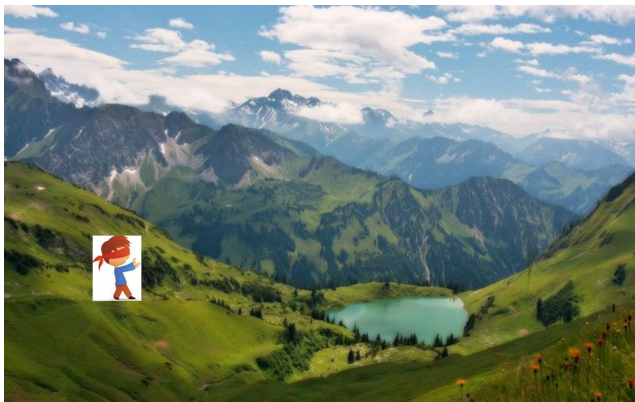


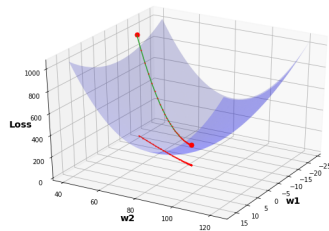
Figura: Analogia do processo de otimização. ¹

¹ http://cs231n.stanford.edu/slides/2016/winter1516_lecture3.pdf

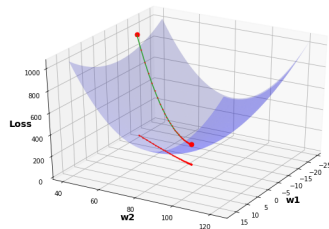
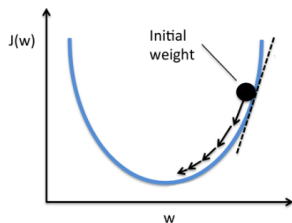
Otimização



- Intuitivamente, o nosso "medidor de altura" é uma função da *loss* em relação aos pesos.



Otimização



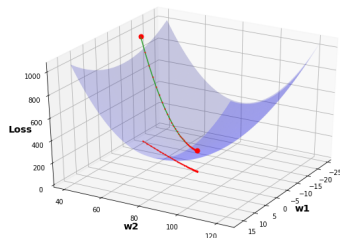
- A direção é dada pela derivada do custo em relação ao peso.
- Para múltiplas dimensões o vetor de derivadas parciais é chamado de **gradiente**.

Otimização



- Em outras palavras, eu consigo alterar os pesos e detectar como a função de perda foi impactada pela minha alteração.

- $$\frac{\partial \mathcal{L}(W, b)}{\partial W_{ij}^{(L)}}$$

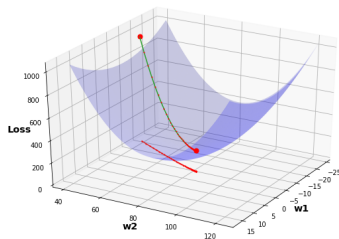


Otimização



- Isso vai indicar:
 - O sinal: se o peso deve aumentar ou diminuir
 - A magnitude: o quanto aquele peso deve ser ajustado.

- $$\frac{\partial \mathcal{L}(W, b)}{\partial W_{ij}^{(L)}}$$



Otimização



- Descida do gradiente "*vanilla*"

$$W_{ij}^{(L)} = W_{ij}^{(L)} - \alpha \frac{\partial \mathcal{L}(W, b)}{\partial W_{ij}^{(L)}}$$

$$b_{ij}^{(L)} = b_{ij}^{(L)} - \alpha \frac{\partial \mathcal{L}(W, b)}{\partial b_{ij}^{(L)}}$$

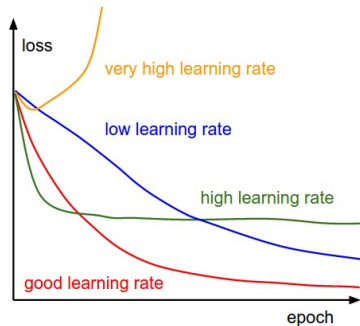
- Sendo α a taxa de aprendizado (*learning rate*)
 - Um fator que interfere na magnitude da alteração dos pesos.

```
for i in range(nb_epochs):  
    params_grad = evaluate_gradient(loss_function, data, params)  
    params = params - learning_rate * params_grad
```

Otimização



- Taxa de Aprendizado: Define o tamanho do "passo" de otimização



Otimizadores



- Gradient Descent (GD)
- Stochastic Gradient Descent (SGD)
- Momentum
- AdaGrad (Adaptive Gradient)
- RMSprop
- Adam

Otimizadores



- Gradient Descent (GD)
 - Calcula o forward e o backward pass para o dataset todo
 - Computa o gradiente conjunto para todas as amostras
 - Extremamente caro

Otimizadores



- Stochastic Gradient Descent (SGD)
 - Computa o gradiente para apenas um **mini-batch** em cada iteração
 - Produz resultados menos precisos que o GD
 - Suscetível a mínimos locais

SGD

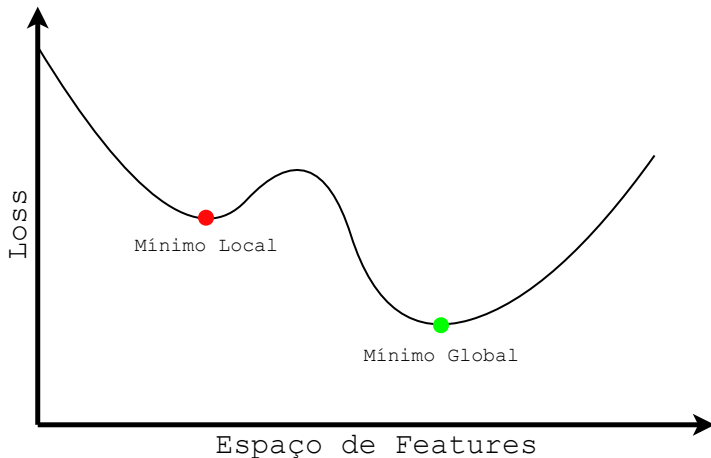


Figura: Exemplo de SGD.

SGD

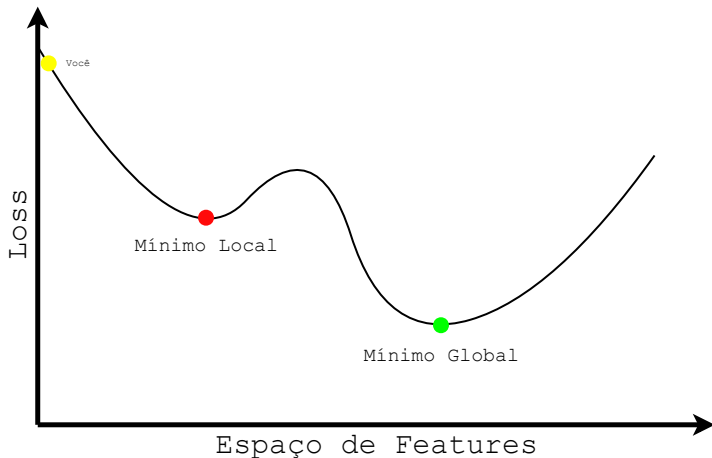


Figura: Exemplo de SGD.

SGD

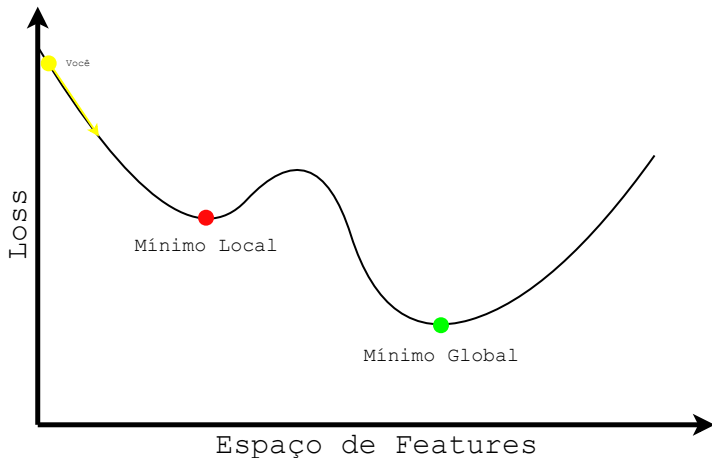


Figura: Exemplo de SGD.

SGD

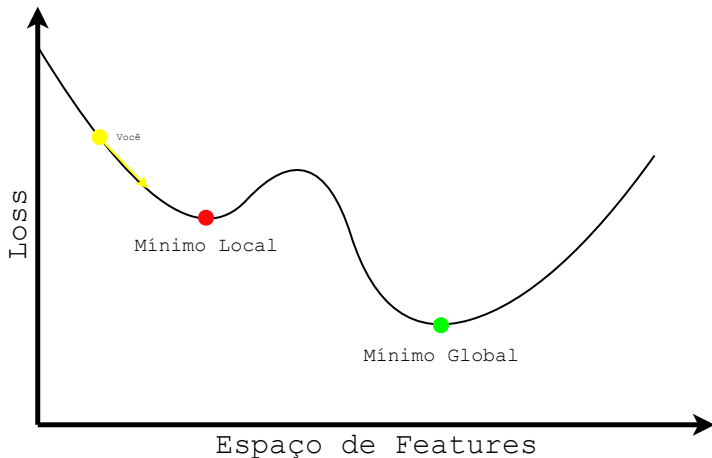


Figura: Exemplo de SGD.

SGD

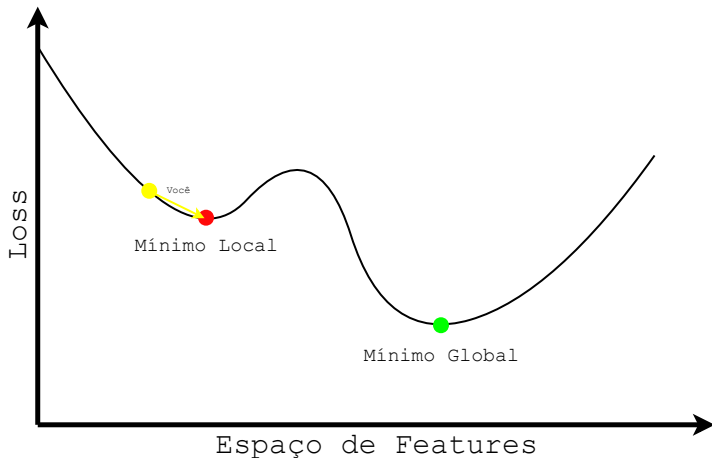


Figura: Exemplo de SGD.

SGD

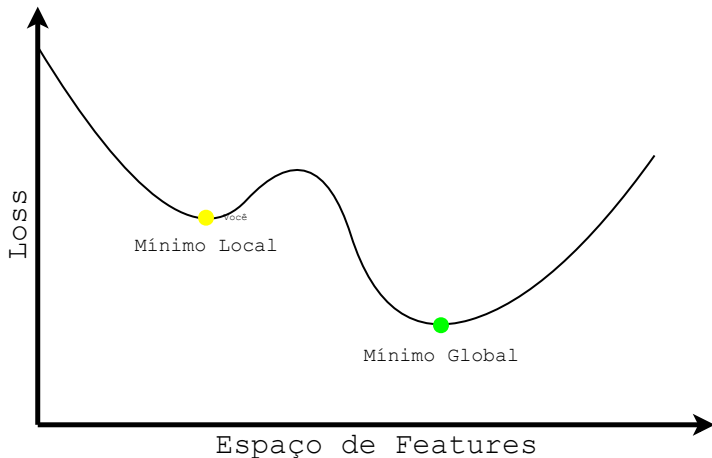


Figura: Exemplo de SGD.

Otimizadores



- Momentum
 - Considera a “velocidade” dos steps da rede
 - Maior robustez a mínimos locais

Momentum

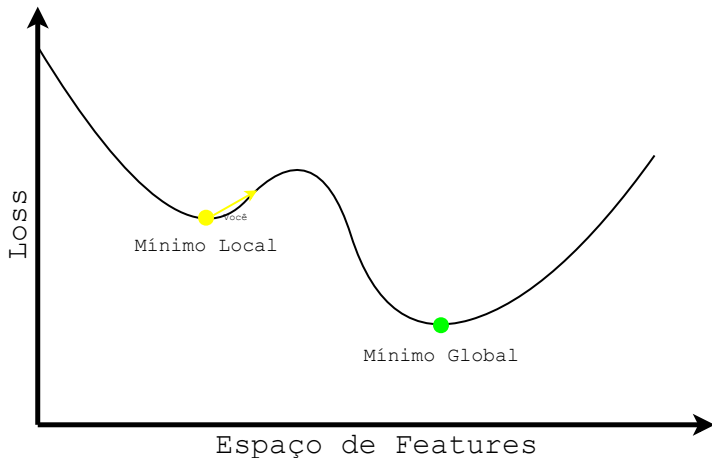


Figura: Exemplo de Momentum.

Momentum

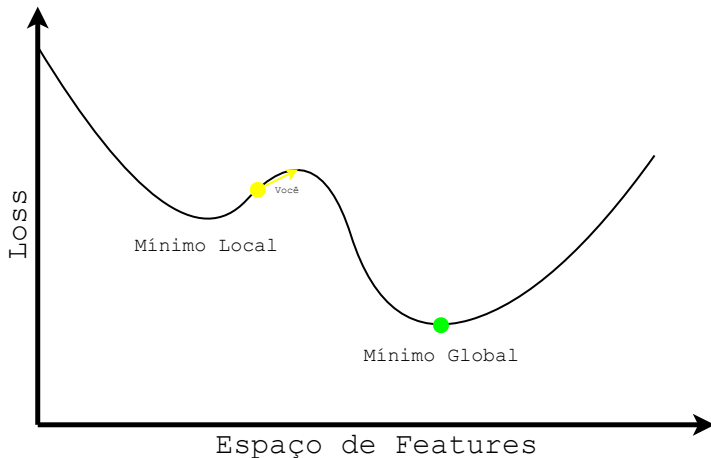


Figura: Exemplo de Momentum.

Momentum

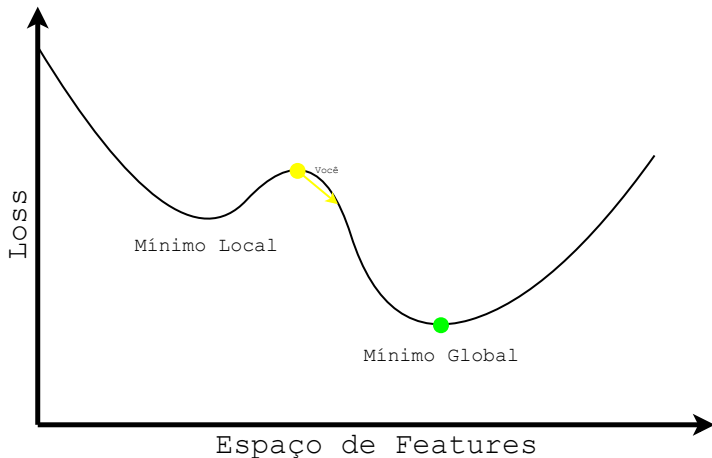


Figura: Exemplo de Momentum.

Momentum

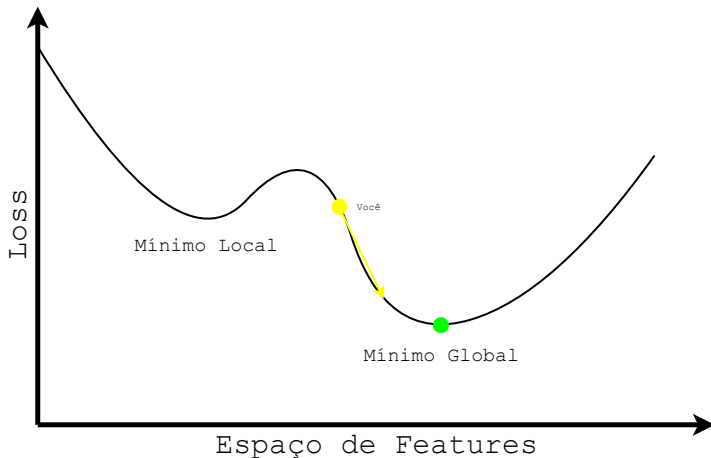


Figura: Exemplo de Momentum.

Momentum

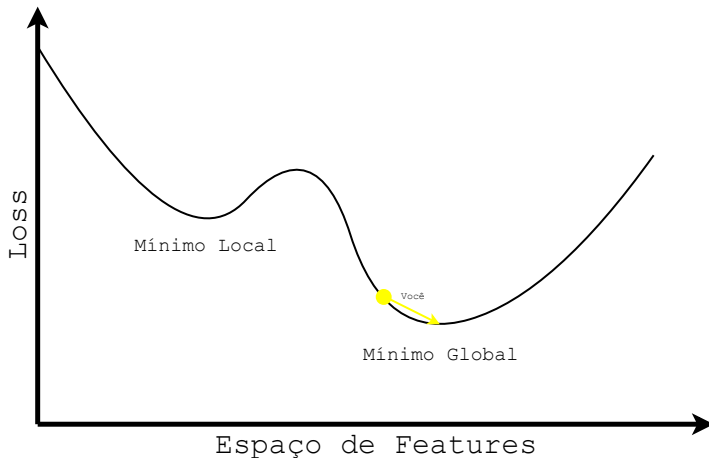


Figura: Exemplo de Momentum.

Momentum

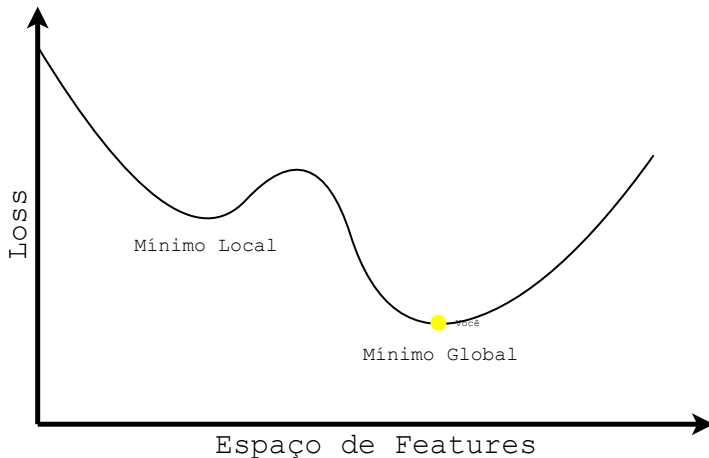


Figura: Exemplo de Momentum.

Otimizadores



- Adaptive (Sub)Gradient (AdaGrad) [3]
 - Velocidade de mudança diferente para cada parâmetro na NN
 - Em geral, para de convergir consideravelmente antes de chegar no Mínimo Global
 - Tende a resultar em underfitting

Otimizadores



- RMSprop
 - Acaba com o problema de underfitting do AdaGrad
 - Fórmula de otimização não publicada
 - Retirada do curso no Coursera do Geoffrey Hinton ¹

¹ http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

Otimizadores



- Adam
 - Junta o melhor do Momentum com o melhor do AdaGrad/RMSprop
 - Estado da Arte atualmente

Otimizadores



Comparação de Otimizadores

Para mais informações sobre otimizadores, visitar o blog post “An overview of gradient descent optimization algorithms”²

²<http://ruder.io/optimizing-gradient-descent/index.html>

Perceptron como classificador linear



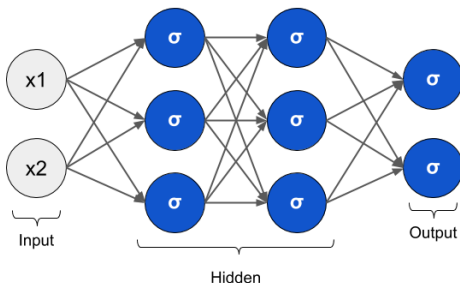
Demo - Classificação Linear

linear_demo.ipynb

Multi-Layer Perceptron



- Além das camadas de entrada e saída, temos também as camadas escondidas (*hidden layers*).

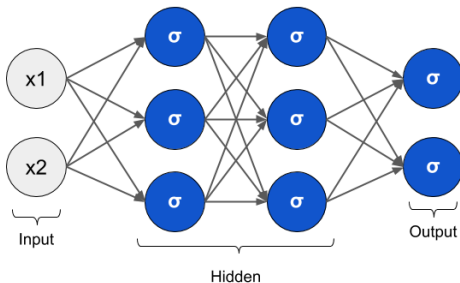


Multi-Layer Perceptron



- Considerando o que já sabemos sobre o perceptron, uma arquitetura multicamada é uma função composta

$$f(f(f(x; W^0, b^0); W^1, b^1); W^2, b^2)$$



Multi-Layer Perceptron



- Considerando o que já sabemos sobre o perceptron, uma arquitetura multicamada é uma função composta

$$f(f(f(x; W^0, b^0); W^1, b^1); W^2, b^2)$$

$$\begin{cases} a^0 = f(x; W^0, b^0) \\ a^1 = f(a^0; W^1, b^1) \\ a^2 = f(a^1; W^2, b^2) \end{cases}$$

Multi-Layer Perceptron



- Teorema de Aproximação Universal ²
 - *"Uma rede neural feed forward com apenas uma camada (escondida) é suficiente para representar qualquer função, mas a camada pode ser inviavelmente grande e pode falhar em aprender e generalizar corretamente."*

² Ian Goodfellow, Deep Learning Book. <http://www.deeplearningbook.org/contents/mlp.html>

Multi-Layer Perceptron



- Teorema de Aproximação Universal ²
 - *"Uma rede neural feed forward com apenas **uma camada (escondida)** é suficiente para representar **qualquer função**, mas a camada pode ser inviavelmente grande e pode falhar em aprender e generalizar corretamente."*

★ O algoritmo de Backpropagation e a arquitetura de neurônios com ativações não-lineares são dois dos principais responsáveis por essa característica.

²Ian Goodfellow, Deep Learning Book. <http://www.deeplearningbook.org/contents/mlp.html>

Perceptron



Treinando um Perceptron

Teorema_da_Aproximação_Universal.ipynb

Multi-Layer Perceptron



Demo - Classificação Linear

nonlinear_demo.ipynb

Multi-Layer Perceptron



- Teorema de Aproximação Universal

Qual desses modelos você acha melhor?

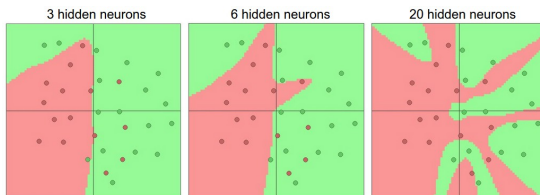


Figura: ConvnetJS demo ³

³ConvnetJS demo <https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

Multi-Layer Perceptron



- Teorema de Aproximação Universal
- A capacidade das redes neurais de aproximar qualquer função contínua (inclusive as mais complexas), a torna muito vulnerável a **overfitting**.

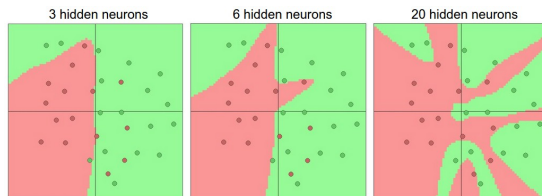


Figura: ConvnetJS demo ³

³ConvnetJS demo <https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

Regularização



- Existem artifícios para controlar a complexidade do modelo.

1 Regularização L2

- Adiciona o termo $\frac{1}{2} \lambda W^2$ à função objetivo para todos os pesos da rede
- λ controla a força da regularização
- Pesos decaem em direção a 0

2 Dropout

- Durante o treinamento, cada neurônio tem uma probabilidade p de estar ativo.
- Neurônios inativos não são atualizados naquela iteração
- Não há dropout em tempo de teste

Regularização



- Efeito de aplicar regularização L2 no modelo com 20 neurônios

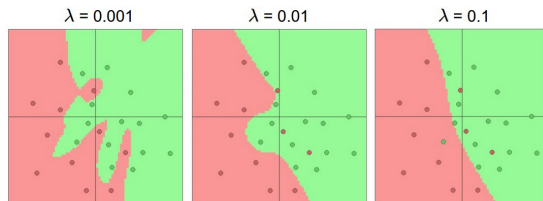


Figura: ConvnetJS demo ³

³ConvnetJS demo <https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

Definindo uma Arquitetura no PyTorch



- O pacote *torch.nn* contém as principais camadas
 - **Convolução/Deconvolução:** Conv1d, Conv2d, Conv3d, ConvTranspose1d, ConvTranspose2d, ConvTranspose3d
 - **Pooling/Unpooling:** MaxPool1d, MaxPool2d, MaxPool3d, MaxUnpool1d, MaxUnpool2d, MaxUnpool3d
 - **Funções de Ativação:** ReLU, Sigmoid, Tanh, Softplus, Softmax, LogSoftmax
 - **Regularização:** Dropout, Dropout2d, Dropout3d
 - **Normalização:** BatchNorm1d, BatchNorm2d, BatchNorm3d, InstanceNorm1d, InstanceNorm2d, InstanceNorm3d
 - **Recorrência:** LSTM, GRU
 - **Densa:** Linear, Bilinear

Dimensões de Entrada e Saída



- Camadas para imagens: Convoluções, Pooling, Batch Normalization 2D...
 - Input: $B \times C_{in} \times H_{in} \times W_{in}$
 - B = batch size
 - C_{in} = número de canais da imagem de entrada
 - H_{in} = height da imagem de entrada
 - W_{in} = width da imagem de entrada
 - Output: $B \times C_{out} \times H_{out} \times W_{out}$
 - C_{out} = número de canais da imagem de saída
 - H_{out} = height da imagem de saída
 - W_{out} = width da imagem de saída

Dimensões de Entrada e Saída



- Camadas Fully Connected: Linear, Softmax...
 - Input: $B \times * \times F_{in}$
 - B = batch size
 - $*$ = qualquer número de dimensões adicionais
 - F_{in} = features de entrada
 - Output: $B \times * \times F_{out}$
 - F_{out} = features de saída

Dimensões de Entrada e Saída



- Muitas camadas fazem operações em tensores de quaisquer dimensões e retornam um tensor do mesmo tamanho da entrada: ReLU, Dropout, Sigmoid, Tanh...
 - Input: $*$
 - $*$ = qualquer número de dimensões
 - Output: $B \times * \times F_{out}$

Definindo uma Arquitetura



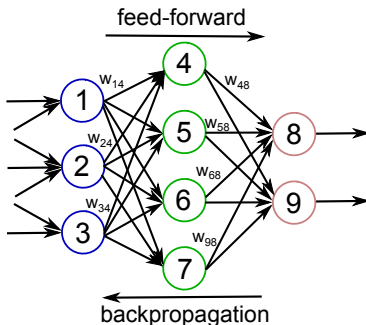
Demo - Architecture

Architecture.ipynb

Redes Neurais Artificiais



- O processo de aprendizado pode ser dividido em dois passos:
 - Feed-forward: recebe os dados e gera a saída final da rede
 - Backpropagation: ajusta os pesos de acordo com o gradiente

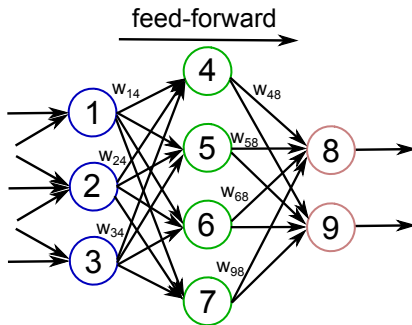


Feed-forward



$$z = \sum_i w_i x_i + b$$

$$a = f(z)$$

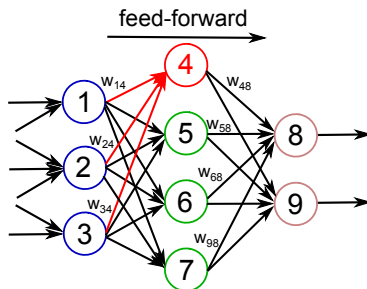


Feed-forward



$$z^4 = W^{14}a^1 + W^{24}a^2 + W^{34}a^3 + b^4$$

$$a^4 = f(z^4)$$

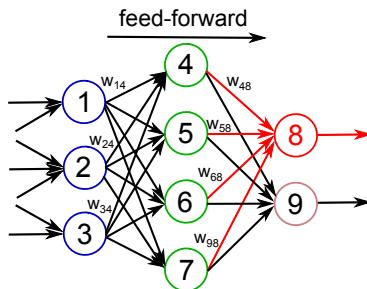


Feed-forward



$$z^8 = W^{48}a^4 + W^{58}a^5 + W^{68}a^6 + W^{78}a^7 + b^8$$

$$a^8 = f(z^8)$$

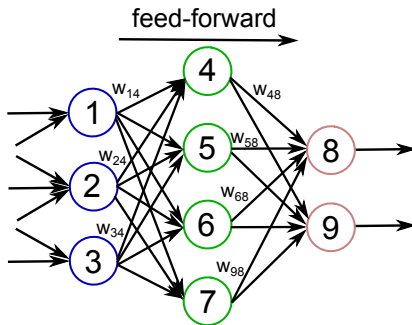


Feed-forward



$$z^{(L+1)} = W^{(L)} a^{(L)} + b^{(L)}$$

$$a^{(L+1)} = f(z^{(L+1)})$$



Backpropagation



- Podemos calcular a velocidade com que o erro muda (usando a *loss function*) à medida que mudamos as saídas dos neurônio
- Para mudar as saídas, precisamos mudar os pesos, o que é feito através do algoritmo de otimização
- Como vimos, precisamos das derivadas parciais
- Backpropagation calcula essas derivadas parciais!

Backpropagation



- Suponhamos:

- Erro quadrático: $\mathcal{L}(W, b; x, y) = \frac{1}{2} \|y - a^{(L)}\|^2$

- Sigmóide como função de ativação: $f(z) = \sigma(z) = \frac{1}{1 + \exp^z}$

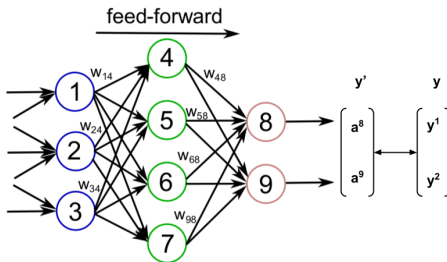
Backpropagation



- Passo a passo, temos:

1 Calcula o erro da rede

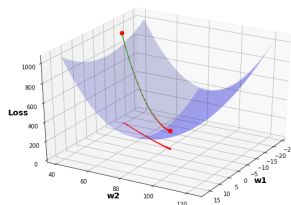
$$\mathcal{L}(W, b; x, y) = \frac{1}{2} \|y - a^{(L)}\|^2$$



Backpropagation



2 Calcula o erro dos neurônios da última camada ⁴



$$\bullet \frac{\partial \mathcal{L}}{\partial w^{(L)}} = a^{(L-1)} a^{(L)} (1 - a^{(L)}) (y - a^{(L)})$$

$$\bullet \delta^{(L)} = a^{(L)} (1 - a^{(L)}) (y - a^{(L)})$$

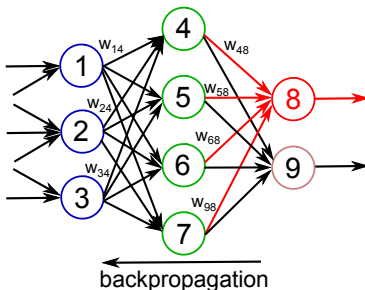
$$\bullet \frac{\partial \mathcal{L}}{\partial w^{(L)}} = a^{(L-1)} \delta^{(L)}$$

⁴ Backpropagation calculus | Appendix to deep learning chapter 3 <https://youtu.be/tleHLnjs5U8>

Backpropagation



- Propagando o erro para neurônios na última camada



- $\delta^8 = a^8(1 - a^8)(y - a^8)$

Backpropagation



- Propagando o erro para neurônios na última camada

- $W^{(L)} = W^{(L)} - \alpha \frac{\partial \mathcal{L}}{\partial W^{(L)}}$

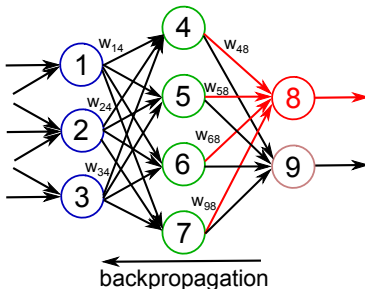
- $\frac{\partial \mathcal{L}}{\partial W^{(L)}} = a^{(L-1)} \delta^{(L)}$

- $W_{48} = W_{48} - \alpha a^4 \delta^8$

- $W_{58} = W_{58} - \alpha a^5 \delta^8$

- $W_{68} = W_{68} - \alpha a^6 \delta^8$

- $W_{78} = W_{78} - \alpha a^7 \delta^8$



Backpropagation



3 Calcula os erros das camadas escondidas

- Mais regra da cadeia!

$$\frac{\partial \mathcal{L}}{\partial W^{(L-1)}}$$

- Iterando em todos os neurônios da camada L , o $\delta^{(L-1)}$ de qualquer neurônio da camada $L - 1$ é dado por:

$$\delta^{(L-1)} = \left(\sum_i \delta_i^{(L)} w_i^{(L)} \right) f'(z^{(L-1)})$$

Backpropagation



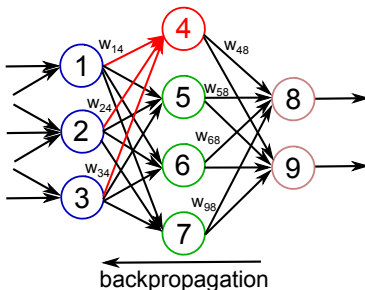
- Propagando para um neurônio de camada escondida

- $\delta^4 = (\delta^8 W_{48} + \delta^9 W_{49}) a^4 (1 - a^4)$

- $W_{14} = W_{14} - \alpha \delta^4 a^1$

- $W_{24} = W_{24} - \alpha \delta^4 a^2$

- $W_{34} = W_{34} - \alpha \delta^4 a^3$



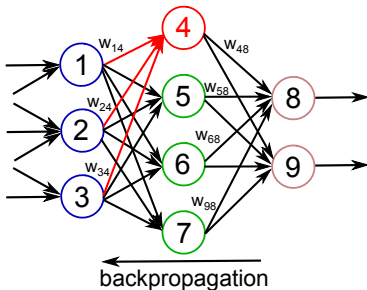
Backpropagation



- Propagando para um neurônio de camada escondida em relação ao bias

- $\delta^4 = (\delta^8 W_{48} + \delta^9 W_{49}) a^4 (1 - a^4)$

- $b_4 = b_4 - \alpha \delta^4 * 1$



Procedimento de Treinamento



- **Configurações Iniciais** (O que sabemos até então)
 - Arquitetura
 - Função de Perda
 - Regularização
 - Otimizador
- Tudo isso é definido experimentalmente (conhecimento prévio pode ajudar bastante) e é diretamente ligado ao problema em questão

Redes Neurais



Atividade - Hiperparâmetros

hyperparameters.ipynb

Procedimento de Treinamento



- Dinâmica **iterativa** de treinamento
 - A otimização é realizada a cada **passo** de treinamento (analogia da montanha).
 - De forma prática que é um passo de treinamento?

Procedimento de Treinamento



Batch

Iteração

Época



Batch

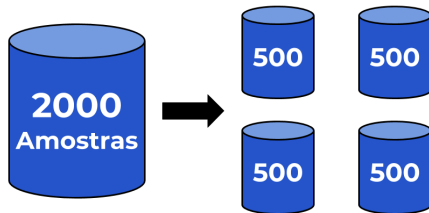


- Subconjunto do dataset.
- Quantidade de amostras em um passo de treinamento.
 - Um passo: Forward + Backward
 - Em outras palavras, quantas amostras o modelo vai ver antes de calcular a loss e otimizar.

Batch



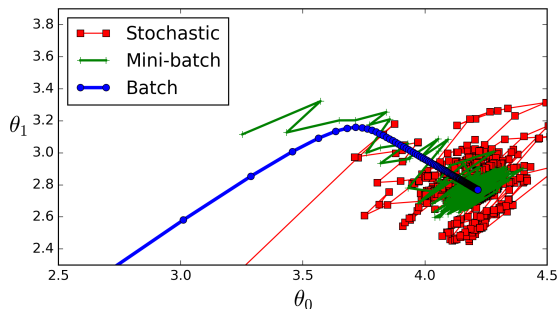
- Por exemplo, dado um dataset de 2000 amostras, poderíamos dividi-lo em batches de 500 amostras.
 - *batch_size* é um hiperparâmetro do treinamento
 - Nesse caso, *batch_size* = 500



Batch



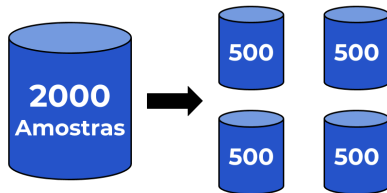
- O tamanho do batch interfere no comportamento de convergência
- Otimização com diferentes tamanhos de batch.



Iteração



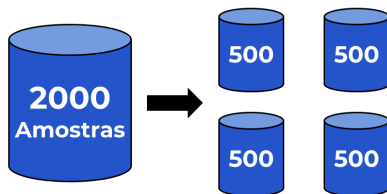
- Efetivamente o passo de treinamento (forward + backward).
- Cada iteração usando *batch_size* número de amostras.
- Número de iterações é diretamente relacionado ao tamanho do batch.
 - No exemplo anterior, teríamos 4 iterações.



Época



- Uma época é completada quando todas as amostras do dataset foram vistas pelo modelo.
- No nosso exemplo, 1 época se passa a cada 4 iterações.



Época



- *num_epochs* também é hiperparâmetro do treinamento.
- **Mas se ao final de uma época o modelo viu todas as amostras do dataset, por que precisamos de mais de uma época?**

Época



- Modelos profundos são *data hungry* (famintos por dados).
- Na maioria dos casos, uma época não é suficiente para o modelo convergir.
 - Quantidade insuficiente de amostras.
- Nessas condições, é necessário apresentar as amostras mais de uma vez ao modelo.

Procedimento de Treinamento



Em resumo

Suponha um dataset de 2000 amostras. Podemos dividir o dataset em **batches** de 500 amostras, fazendo com que sejam necessárias 4 **iterações** para completar 1 **época**.

Data Loader no PyTorch



- Classes *Dataset* e *DataLoader*
 - *DataLoader*: normalmente usada a versão original
 - *Dataset*: normalmente customizada
- Subclasses da classe *Dataset* possuem duas funções principais:
 - `make_dataset()`
 - `__getitem__()`

Data Loader no PyTorch



Demo - Data Loader

Data_Loader.ipynb

Procedimento de Treinamento



- Arquitetura de DNN
- Otimizador
- Data Loader
- Loss Function

Procedimento de Treinamento



Exercício Prático - Training Procedure

MLP_Classification_Olivetti.ipynb

References

- [1] Aleksandra C.
How is a neuron adapted to perform its function?, 2018.
- [2] Fractal Foundation.
Fractal neurons, 2018.
- [3] John Duchi, Elad Hazan, and Yoram Singer.
Adaptive subgradient methods for online learning and stochastic optimization.
[Journal of Machine Learning Research](#), 12(Jul):2121–2159, 2011.

